

Escuela Politécnica Nacional



Informe del Proyecto Bimestral II de Métodos Numéricos

Integrantes:

Wellington Barros

Hodalys López

Josué Mantuano

Jonathan Paredes

Proyecto Impresora 3D

Fecha de entrega: 11 de febrero del 2025

1. Introducción

El presente informe detalla la implementación y análisis de una simulación de una impresora 3D, en el lenguaje de programación Python. Las impresoras 3D son dispositivos electrónicos que fabrican objetos tridimensionales teniendo como guía modelos digitales, en estas se depositan materiales siguiendo una trayectoria previamente definida capa por capa hasta lograr una figura prediseñada.

La simulación se desarrolla en un ambiente bidimensional, es decir, que la impresión se simula en dos dimensiones representadas en el plano como (x,y), la impresora dibuja una trayectoria del cabezal en una superficie plana a partir de varios archivos SVG que son cargados para servir como plano en el cual la impresora dibujará el modelo siguiendo una trayectoria con perspectiva simplificada. Esta implementación no considera aspectos importantes como la cantidad de material, temperatura o adhesión de las capas, sino que se enfoca en la trayectoria del cabezal y la optimización del movimiento, además que el objetivo del proyecto es evaluar el comportamiento de la impresión con parámetros como la resolución y velocidad en la simulación mediante animaciones. Las impresoras 3D son un invento que revolucionaron el área industrial, medicina y diseño de productos, pues con la gran capacidad de creación de objetos funcionales Innovó el mercado. Al implementar los archivos SVG lo que se busca es dar una utilidad de los diseños predefinidos transformándolos en trayectorias de impresión simulables. La disposición de los puntos dentro del contorno del modelo influye en la calidad de la impresión al final de esta, junto con el tiempo que se ocupa para completar todo el proceso.

2. Metodología

2.1 Descripción de la solución

En el programa se implementa una solución que se basa en el uso de Python y varias bibliotecas para poder construir una interfaz gráfica con la cual el usuario pueda interactuar con los parámetros de la simulación. El flujo del programa y su funcionamiento está basado en varios puntos.

1. Selección de un archivo SVG y definición de los parámetros principales (resolución y velocidad) de la impresión.
2. Se extrae el contorno del modelo SVG y los puntos dentro del área son filtrados.
3. Se hace un ordenamiento en forma de zigzag para la simulación del trayecto del cabezal de la impresora 3D.
4. En pantalla se muestra la trayectoria de los puntos puestos en tiempo real.

5. Control en tiempo real de la velocidad de la impresión mediante una barra deslizante, permitiendo a los usuarios modificar la velocidad de la simulación mientras se ejecuta

Para generar una solución al proyecto se utilizó bibliotecas para el procesamiento de gráficos y su correcta simulación visual:

- Tkinter: Biblioteca necesaria para la interfaz gráfica con la que interactúa el usuario, además de que permite la selección del archivo SVG y la selección de los parámetros para la simulación
- Matplotlib: Visualización de la trayectoria del cabezal de la impresora y la simulación de esta, además de permitir el manejo de contornos y la verificación de puntos dentro de las figuras.
- NumPy: Cálculos matemáticos necesarios para la manipulación de matrices de los puntos
- Time: Biblioteca utilizada para llevar un control del tiempo de la simulación, junto con la actualización de cada frame en la animación.
- Os: Gestión y lectura de archivos dentro del sistema, facilitando la selección de modelos SVG almacenados en una carpeta específica.
- Xml.dom.minidom: Biblioteca necesaria para leer y extraer los contornos de los modelos SVG.

2.2 Desarrollo matemático

La simulación de la impresora 3D implementado en el código se basa en varios conceptos matemáticos que permiten la conversión y visualización de modelos SVG. A continuación, se detallan algunas de las fórmulas que se han utilizado:

1. Conversión de Círculos a Polígonos

Para convertir un círculo en un polígono, se utiliza la representación paramétrica del círculo. Las ecuaciones son comúnmente empleadas para expresar la trayectoria de un punto en movimiento, donde el parámetro, a menudo, es el tiempo, pero en este caso se interpreta como un ángulo.

La representación paramétrica de un círculo se puede expresar mediante las siguientes fórmulas:

$$x = cx + r \cdot \cos \theta$$

$$y = cy + r \cdot \sin \theta$$

Donde

$$(cx, cy) = \text{Coordenadas del centro del círculo}$$

$r = \text{radio del círculo}$

$\theta = \text{parámetro que toma valores en el intervalo } [0, 2\pi]$

Geométricamente, θ se interpreta como el ángulo que va desde el centro (cx, cy) hasta el punto (x, y) y el eje x positivo.

2. Cálculo de tiempo de simulación

El tiempo de simulación se calcula utilizando la diferencia entre el tiempo actual y el tiempo de inicio:

$$T_{\text{simulación}} = T_{\text{actual}} - T_{\text{inicio}}$$

3. Generación de malla de puntos

Se utiliza `np.arange` para generar secuencias de puntos en los ejes x e y , y luego `np.meshgrid` para crear una malla de coordenadas.

$$x = [x_{\min}, x_{\min} + \text{resolución}, x_{\min} + 2 \cdot \text{resolución}, \dots, x_{\max}]$$

$$y = [y_{\min}, y_{\min} + \text{resolución}, y_{\min} + 2 \cdot \text{resolución}, \dots, y_{\max}]$$

Luego, se combina en una malla:

$$X, Y = \text{np.meshgrid}(x, y)$$

4. Control de velocidad del código

En la implementación, se controla la velocidad utilizando retardos en la ejecución de cada iteración.

El código ajusta el tiempo entre cada actualización de la trayectoria utilizando la función `time.sleep()`, asegurando que la simulación siga la velocidad deseada:

$$\text{target}_{\text{delay}} = \frac{1}{V}$$

2.3 Diagrama de flujo/pseudocódigo

// Programa para simular la impresión 3D a partir de archivos SVG

Proceso SimuladorImpresion3D

Definir archivoSVG Como Cadena

Definir resolución, velocidad Como Real

// Pedir al usuario el archivo SVG

Escribir "Ingrese el nombre del archivo SVG (sin extensión):"

Leer archivoSVG

```
// Pedir la resolución y velocidad
```

```
Escribir "Ingrese la resolución de impresión (mm):"
```

```
Leer resolucion
```

```
Escribir "Ingrese la velocidad de impresión:"
```

```
Leer velocidad
```

```
// Cargar contornos desde el archivo SVG
```

```
Definir contornos Como Lista
```

```
contornos <- CargarContornosSVG(archivoSVG + ".svg")
```

```
// Validar si se cargaron correctamente
```

```
Si Longitud(contornos) = 0 Entonces
```

```
    Escribir "Error: No se pudieron cargar los contornos desde el archivo."
```

```
    FinProceso
```

```
FinSi
```

```
// Simular impresión 3D
```

```
SimularImpresion(contornos, resolucion, velocidad)
```

```
FinProceso
```

```
// Función para cargar contornos desde un archivo SVG
```

```
Funcion CargarContornosSVG(archivo)
```

```
    Definir contornos Como Lista
```

```
    contornos <- []
```

```
// Abrir archivo
```

```
Definir linea Como Cadena
```

```
archivoSVG <- AbrirArchivo(archivo)
```

```
// Leer líneas del archivo
```

```
Mientras NoEsFinDeArchivo(archivoSVG) Hacer
```

```
    linea <- LeerLinea(archivoSVG)
```

```
    Si Contiene(linea, "<polygon") Entonces
```

```
        Definir puntos Como Lista
```

```
        puntos <- ExtraerPuntosDeLinea(linea)
```

```
        AgregarElemento(contornos, puntos)
```

```
    FinSi
```

```
    Si Contiene(linea, "<circle") Entonces
```

```
        Definir cx, cy, r Como Real
```

```
        cx <- ExtraerValor(linea, "cx")
```

```
        cy <- ExtraerValor(linea, "cy")
```

```
        r <- ExtraerValor(linea, "r")
```

```
        Definir circulo Como Lista
```

```
        circulo <- ConvertirCirculoAPoligono(cx, cy, r)
```

```
        AgregarElemento(contornos, circulo)
```

```
    FinSi
```

```
FinMientras
```

```
CerrarArchivo(archivoSVG)
```

```
Retornar contornos
```

```
FinFuncion
```

```
// Función para convertir un círculo en un polígono con puntos
```

```
Funcion ConvertirCirculoAPoligono(cx, cy, r)
```

```
  Definir puntos Como Lista
```

```
  puntos <- []
```

```
  Definir i, angulo, x, y Como Real
```

```
  Para i <- 0 Hasta 100 Con Paso 1 Hacer
```

```
    angulo <- (2 * 3.1416 * i) / 100
```

```
    x <- cx + r * Cos(angulo)
```

```
    y <- cy + r * Sen(angulo)
```

```
    AgregarElemento(puntos, (x, y))
```

```
  FinPara
```

```
  Retornar puntos
```

```
FinFuncion
```

```
// Función para simular la impresión 3D
```

```
Funcion SimularImpresion(contornos, resolucion, velocidad)
```

```
  Definir puntosImpresion Como Lista
```

```
  puntosImpresion <- GenerarPuntosImpresion(contornos, resolucion)
```

```
  // Simular movimiento de impresión
```

```
  Definir i Como Entero
```

```
  Para i <- 0 Hasta Longitud(puntosImpresion) - 1 Con Paso 1 Hacer
```

```
    Escribir "Imprimiendo punto ", i + 1, " de ", Longitud(puntosImpresion)
```

```
    Esperar(1 / velocidad)
```

```
  FinPara
```

Escribir "Impresión completada."

FinFuncion

// Función para generar los puntos de impresión dentro del contorno

Funcion GenerarPuntosImpresion(contornos, resolucion)

Definir puntos Como Lista

puntos <- []

Definir x, y Como Real

// Definir los límites del área de impresión

Definir minX, maxX, minY, maxY Como Real

minX <- MinimoX(contornos)

maxX <- MaximoX(contornos)

minY <- MinimoY(contornos)

maxY <- MaximoY(contornos)

// Recorrer la región con la resolución dada

Para x <- minX Hasta maxX Con Paso resolucion Hacer

Para y <- minY Hasta maxY Con Paso resolucion Hacer

Si PuntoDentroContorno(x, y, contornos) Entonces

AgregarElemento(puntos, (x, y))

FinSi

FinPara

FinPara

Retornar puntos

FinFuncion

Explicación del Pseudocódigo

SVGLoader

- Se encarga de leer archivos SVG y extraer los contornos de los modelos.
- Convierte círculos en polígonos mediante una aproximación con puntos.
- Lee polygon, circle y path desde el archivo SVG.

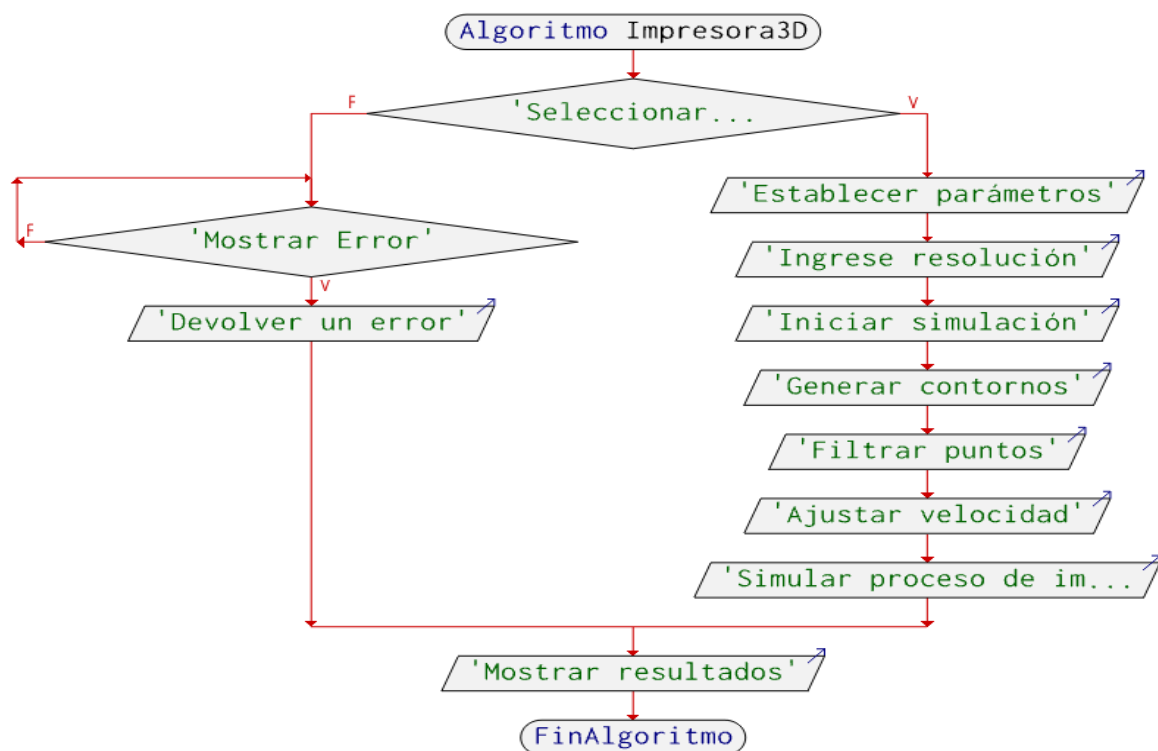
Simulator

- Simula la impresión del modelo SVG en una ventana gráfica usando Matplotlib.
- Filtra los puntos dentro del contorno exterior y fuera de los huecos.
- Genera una trayectoria de impresión basada en líneas ordenadas.

App

- Maneja la interfaz gráfica con Tkinter.
- Permite seleccionar un archivo SVG, definir los parámetros como resolución y velocidad.
- Inicia la simulación cuando el usuario da clic en “Comenzar”.

Diagrama de flujo



2.4 Detalles importantes de implementación

Interfaz gráfica con Tkinter:

- Tkinter es implementado para generar una interfaz gráfica.
- Entrada del modelo SVG.
- Configuración de parámetros importantes como la resolución del modelo a simular
- Manipulación de velocidad de la simulación con control deslizante en tiempo real.

Procesamiento del modelo SVG:

- Uso de minidom para extraer y leer los archivos SVG y extraer los contornos del modelo.
- Extracción de polígonos, círculos y rutas (<polygon>, <circle>, <path>) y se convierten en listas de coordenadas.
- Almacenamiento de los puntos del contorno exterior y los de los huecos internos, si los hay.

Generación de trayectorias:

- Matplotlib.path.Path.contains_points filtra los puntos dentro del contorno del modelo y evita los que están fuera.
- Organización de puntos en zigzag para simular el movimiento del cabezal.
- Los puntos almacenados en la matriz se convierten en una trayectoria ordenada para la simulación.

Animación y visualización:

- Uso de Matplotlib para representar el contorno y los puntos impresos.
- Animación cuadro por cuadro que simula el proceso de impresión.
 - Puntos naranjas representan posiciones impresas
 - Líneas grises representan la trayectoria del cabezal
 - El punto verde representa la posición actual del cabezal.

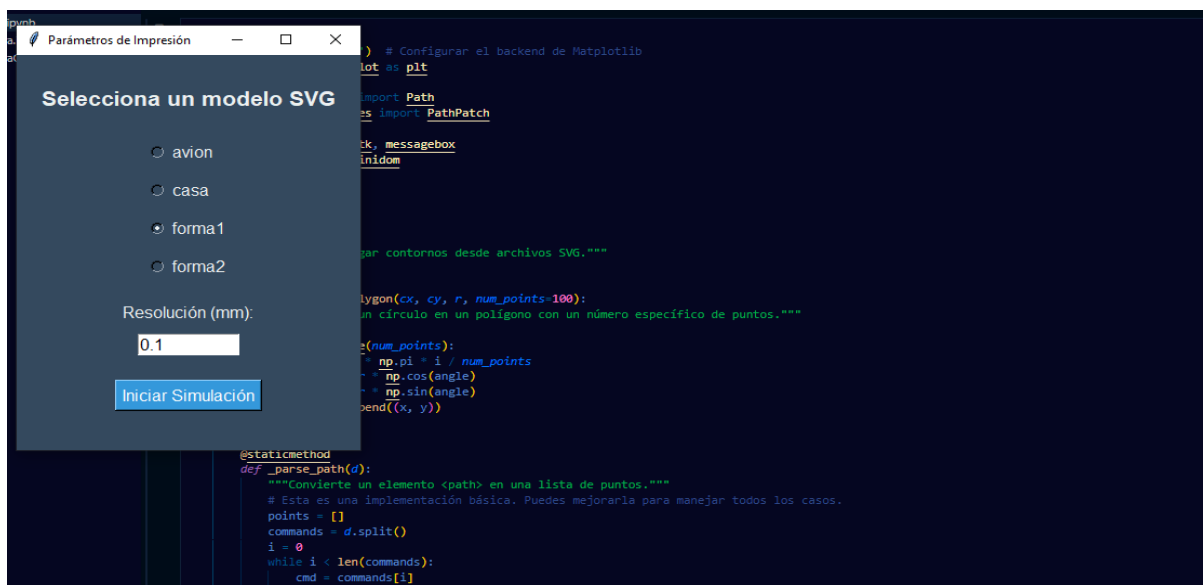
3. Resultados

Se desarrolló una interfaz gráfica intuitiva y moderna utilizando Tkinter.

La interfaz incluye:

- Pantalla de bienvenida con descripción del proyecto

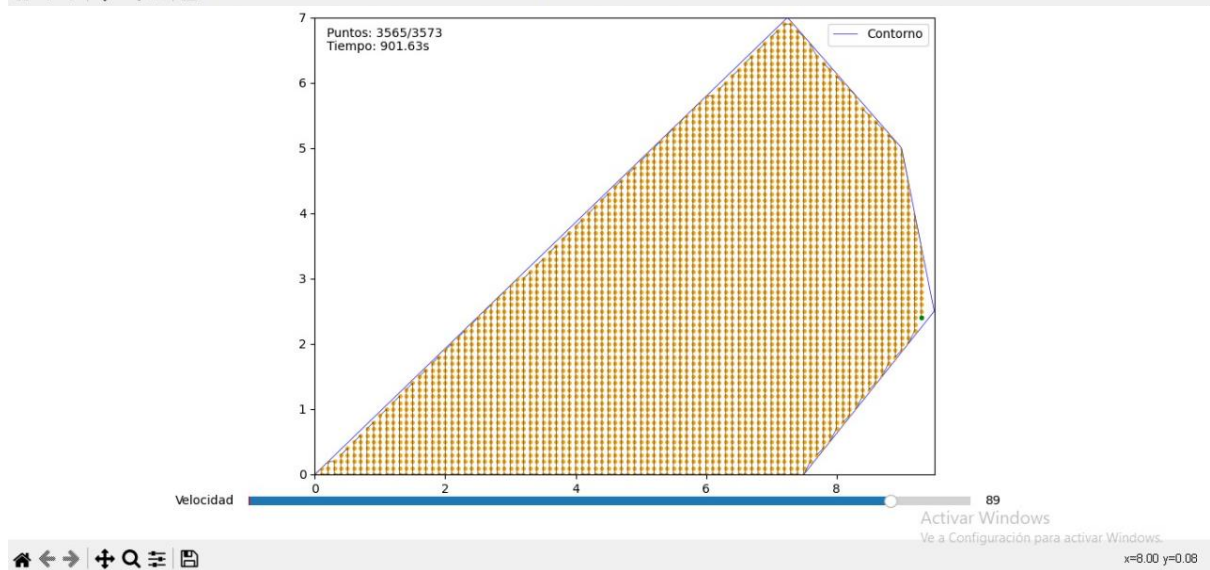
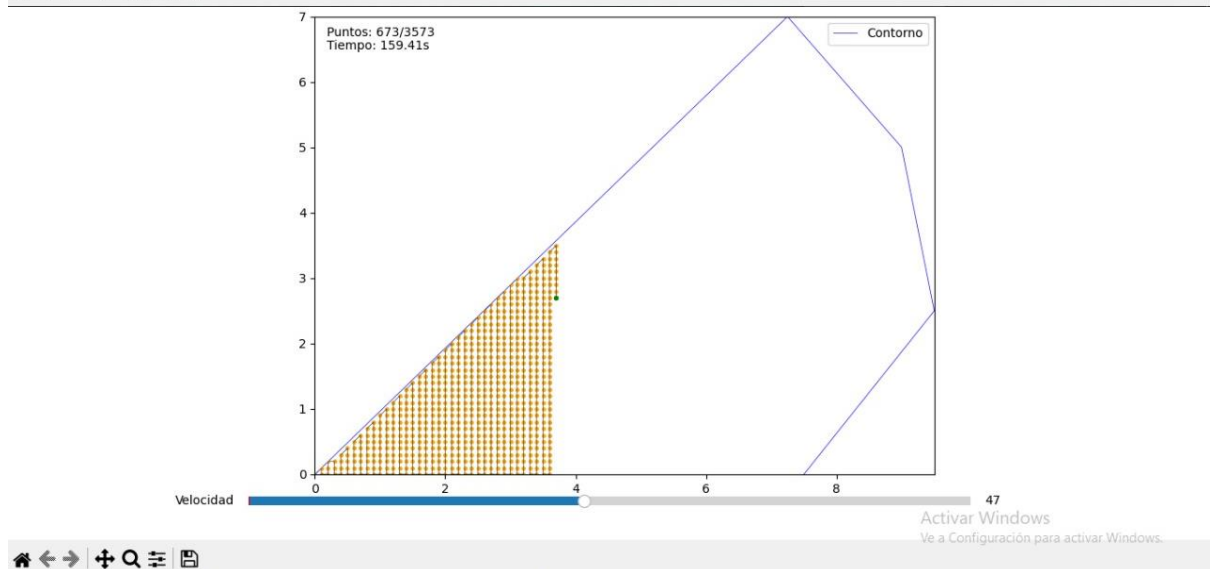
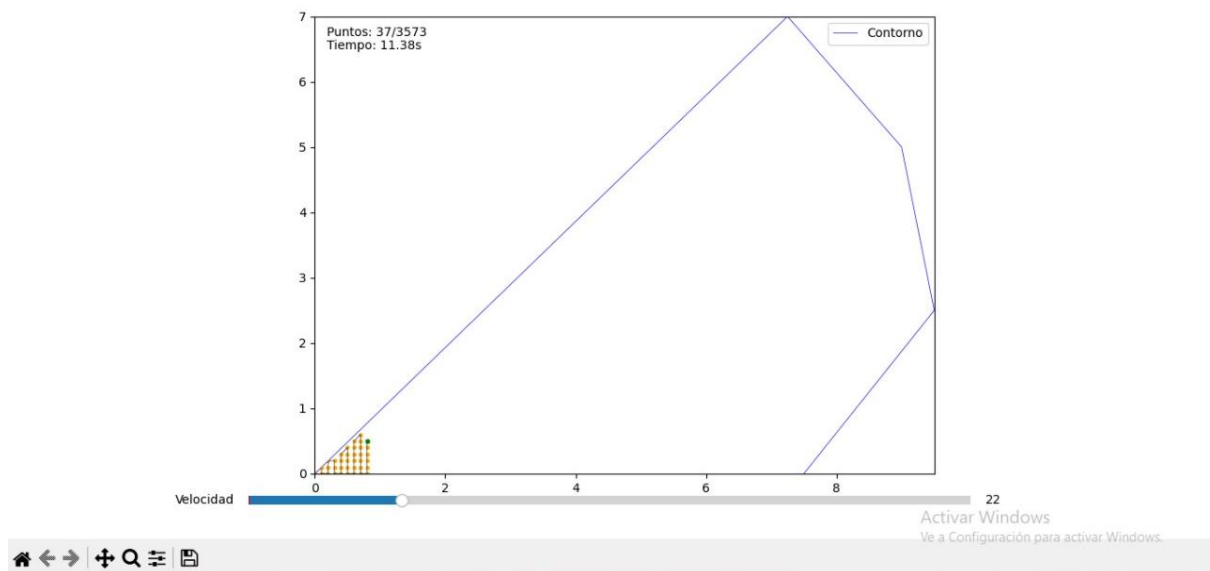
- Ventana de configuración de parámetros
- Visualización en tiempo real de la simulación
- Control de velocidad de simulación mediante slider



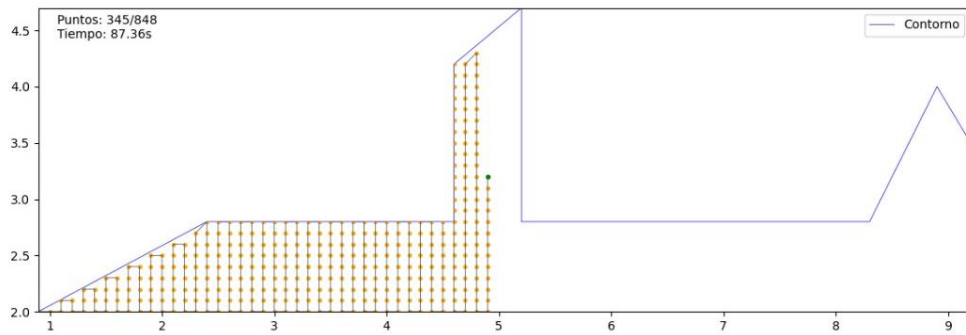
En esta sección se presentan los resultados obtenidos tras seleccionar el modelo SVG. Las imágenes muestran el proceso de simulación en el que se observa el llenado progresivo de la región definida por los parámetros de velocidad y tiempo.

A medida que avanza la simulación, se puede notar cómo el área sombreada aumenta progresivamente, indicando el crecimiento de la región de interés.

Figura 1:



Avión:



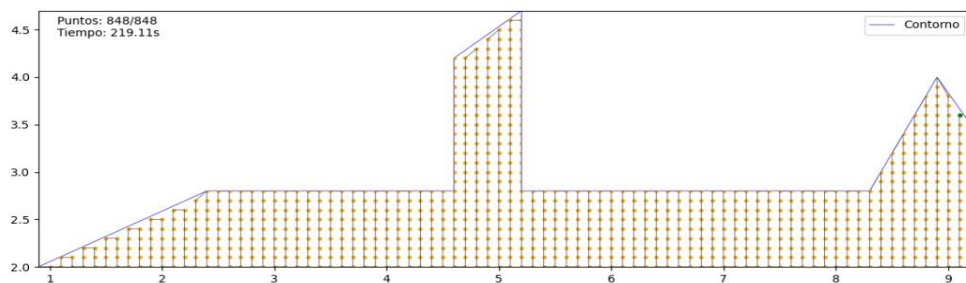
Velocidad 11

Activar Windows

Ve a Configuración para activar Windows.



x=3.575 y=2.534



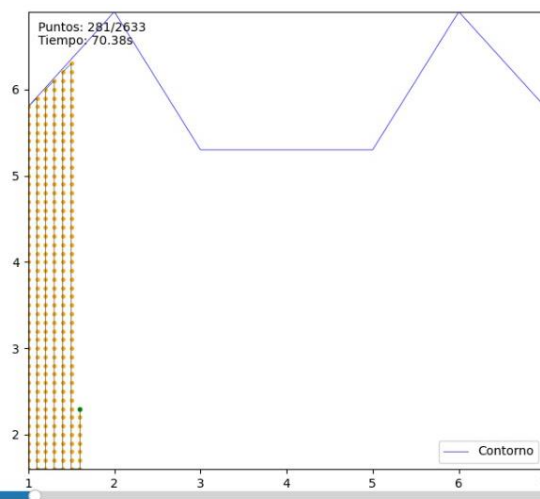
Velocidad 89

Activar Windows

Ve a Configuración para activar Windows.



Casa:

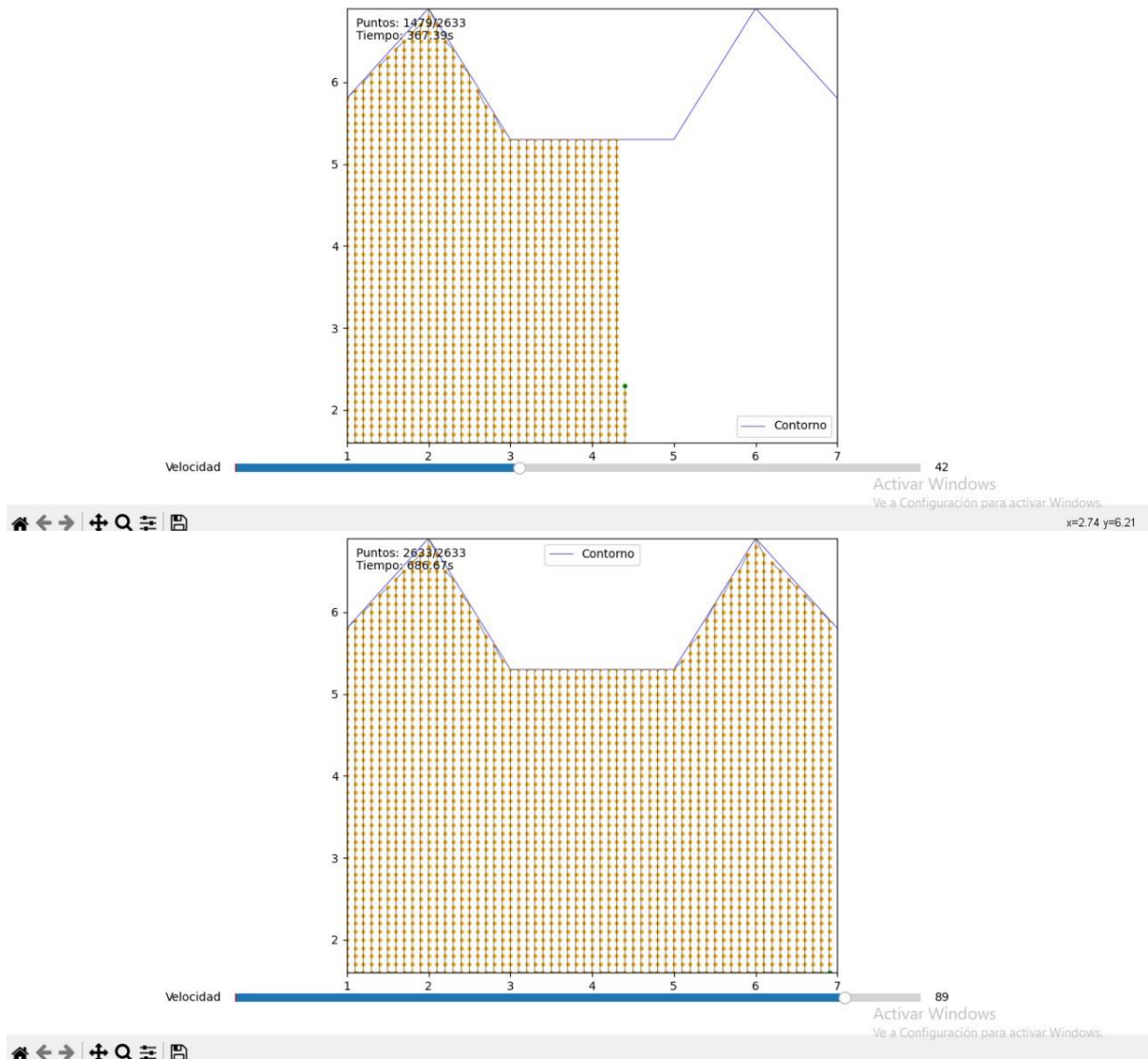


Velocidad 18

Activar Windows

Ve a Configuración para activar Windows.





Estos resultados reflejan el correcto funcionamiento del algoritmo, asegurando que la impresión o modelado de la estructura se lleva a cabo de manera ordenada y eficiente.

El simulador maneja modelos SVG de diferentes complejidades y la velocidad de simulación es ajustable en tiempo real.

4. Conclusiones

Los resultados obtenidos demuestran que el algoritmo implementado funciona correctamente, permitiendo un llenado progresivo y estructurado del área definida. La simulación muestra cómo la impresión avanza de manera ordenada, asegurando precisión y coherencia en el proceso.

La ejecución del código confirma que el modelo utilizado es eficiente en la generación de patrones de impresión. La progresión del llenado indica que el sistema optimiza la

cobertura del área dentro del contorno establecido, minimizando errores y maximizando la precisión.

Al implementar una representación gráfica nos permite verificar que no hay interrupciones ni inconsistencias en el proceso, lo que sugiere que la estrategia implementada es confiable y replicable en diferentes escenarios.

El comportamiento del sistema demuestra que la metodología aplicada es efectiva para realizar un llenado sistemático del área sin desperdiciar recursos computacionales.